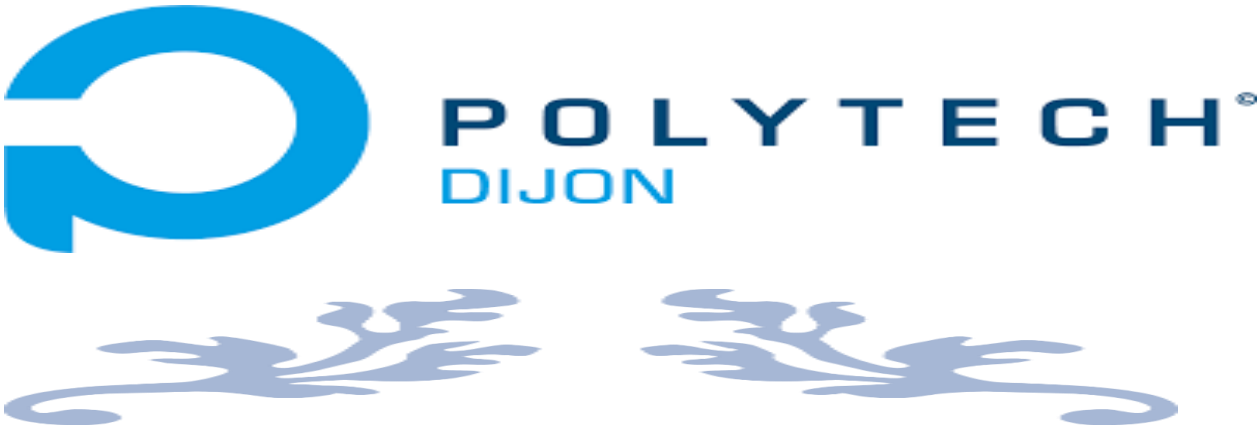




PIC24 PROJECT

World Report on Temperature Monitoring System Using PIC24FJ128GB610



Written by:

- MAMBO MOTSOU JUNIOR DEOGRACIAS

In today’s world, temperature monitoring plays a crucial role in various applications, from environmental control to industrial processes. This project seeks to create an embedded system capable of measuring and recording temperature data with high precision. By utilizing the PIC24FJ128GB610 microcontroller, the system not only captures real-time temperature but also associates each reading with a timestamp. The collected data is stored in EEPROM memory, enabling the retrieval of historical temperature measurements. With the integration of an LCD display, users can easily access both current and past temperature records. This project combines key embedded system concepts such as sensor interfacing, real-time clock handling, and data storage in a simple and efficient manner, aimed at providing a reliable solution for temperature monitoring.

I. Introduction

The goal of this project is to design and implement a temperature monitoring system using the PIC24FJ128GB610 microcontroller. The system is intended to measure ambient temperature in real-time, store these measurements along with precise timestamps (in minutes and seconds), and allow for historical data retrieval and display. The temperature data is captured using an ADC connected to a temperature sensor and then stored in EEPROM for later access. The system provides users with the ability to view both real-time temperature readings and historical data through an LCD interface, offering valuable insights into temperature trends over time. Ultimately, this project aims to demonstrate the integration of various embedded system components such as temperature sensing, real-time clock management, data storage, and display, in a compact and efficient solution.

1. Temperature (ADC module) Reading

In the first part of the project, the goal is to read the temperature using the TC1047A sensor, configure the microcontroller's ADC for accurate measurements, and display the data on an LCD.

1. Sensor Port Identification: The TC1047A sensor outputs an analog signal corresponding to temperature. The sensor is connected to an ADC input port on the PIC24FJ1024GB610 RB4.
2. Configuring the ADC: Using MPLAB Code Configurator (MCC), the ADC module is set up to sample the sensor's analog voltage, convert it to a digital value, and store the result for processing. Configuration includes selecting the correct input pin, setting ADC resolution, adjusting clock and sample time, and enabling interrupts to handle conversions efficiently.
3. Interrupt Handling: ADC interrupts are used to read sensor data without continuous polling. Once the ADC conversion is complete, the interrupt triggers the reading of the digital value, which is then processed and displayed. The ADC interrupt flag is checked and cleared after each conversion.
4. Data Conversion and Display: The raw ADC result is converted into temperature using the TC1047A transfer function, and displayed as a readable string (e.g., "25.2°C") on the LCD screen.

Challenges:

The challenges in this part of the project included ensuring correct ADC configuration, managing interrupt handling, and accurately converting the ADC result to a temperature. However, these challenges were manageable due to previous experience with similar tasks in TD/TP classes, where ADC configuration, sensor data acquisition, and interrupt handling were practiced.

2. Enhancing the System with RTCC Integration

In the second part of the project, we enhance the system by integrating the Real-Time Clock Calendar (RTCC) module, which allows us to record

temperature readings along with the corresponding date and time. This addition makes the measurements more meaningful and traceable.

Since we already worked with the RTCC module in class, this part of the project was straightforward. The RTCC module on the PIC24FJ1024GB610 was configured to provide accurate time and date information. We used MCC to configure the RTCC, set the clock, and enable interrupts for time updates.

The main task involved synchronizing the RTCC data with the temperature readings. Whenever a temperature reading is taken, the current time from the RTCC is fetched, and both the temperature and time are displayed on the LCD (“25.2°C 12:30:45"). This makes the system not only capable of measuring temperature but also provides a timestamp for each reading, adding greater functionality to the system.

The process of configuring the RTCC was like the previous part, requiring only minor adjustments to the existing code. The real challenge here was integrating the time data into the system's output without affecting the timing of temperature measurements. However, this was relatively easy thanks to prior class exercises on RTCC setup and time handling.

3. Part 3: Utilizing EEPROM for Historical Temperature Storage

The third part of the project involves storing the last 10 temperature measurements in an EEPROM (25LC256) memory module. This allows the system to retain a history of the measurements, which can be retrieved and displayed on a user interface (UI). This section can be divided into several key components:

3.1 User Interface (UI)

The user interface **(UI)** plays a crucial role in this part of the project. The system is designed to allow users to interact with the historical data stored in the EEPROM. The UI provides a way for the user to:

- View the most recent temperature readings stored in memory.
- Retrieve and display the historical temperature measurements (the last 10 readings) along with their corresponding timestamps.

The UI will be displayed on the LCD screen, where users can choose an option to view the data. The interface will feature simple navigation to retrieve the stored temperature data one by one or display the full list of the last 10 temperature readings.

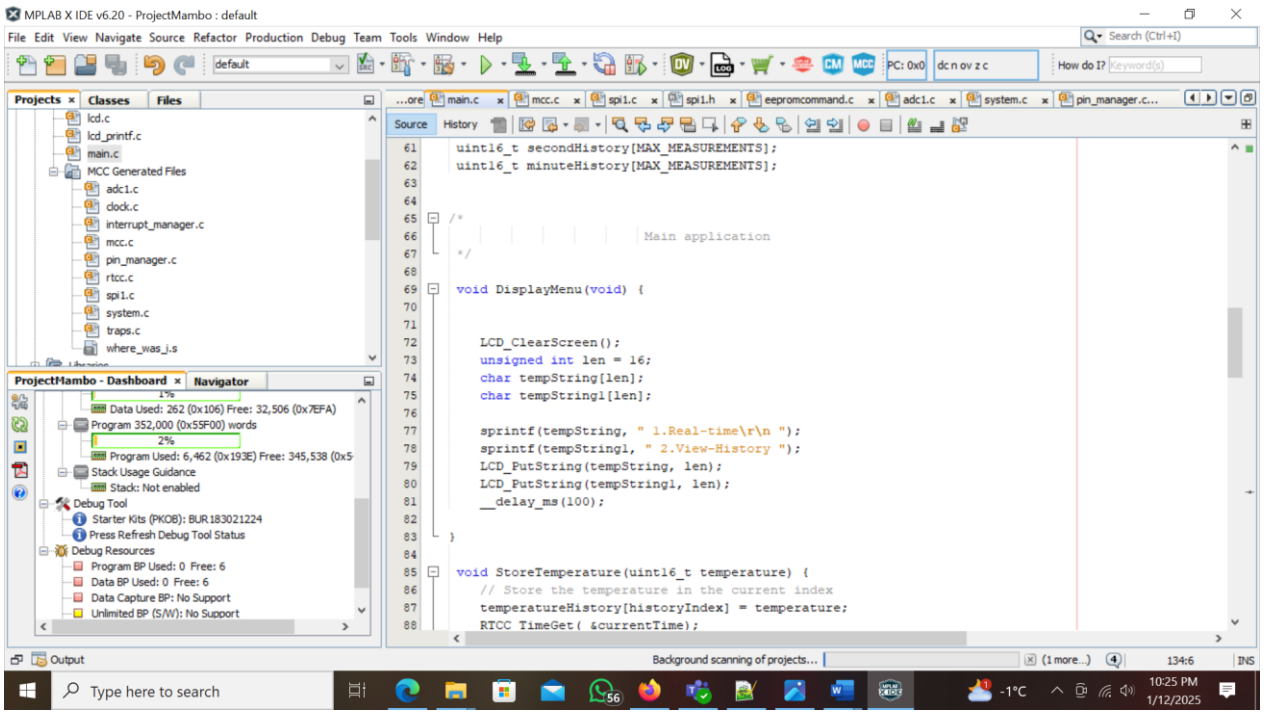


Figure 1:UI code

The DisplayMenu function displays a simple menu on the LCD with two options: "1. Real-time" and "2. View History." It clears the screen, formats the strings using sprintf, and displays them using LCD_PutString. A 100ms delay ensures the menu is visible before the program continues. This function provides a user-friendly interface for navigating between real-time temperature readings and historical data.



Figure 2:Ui Physical presentation

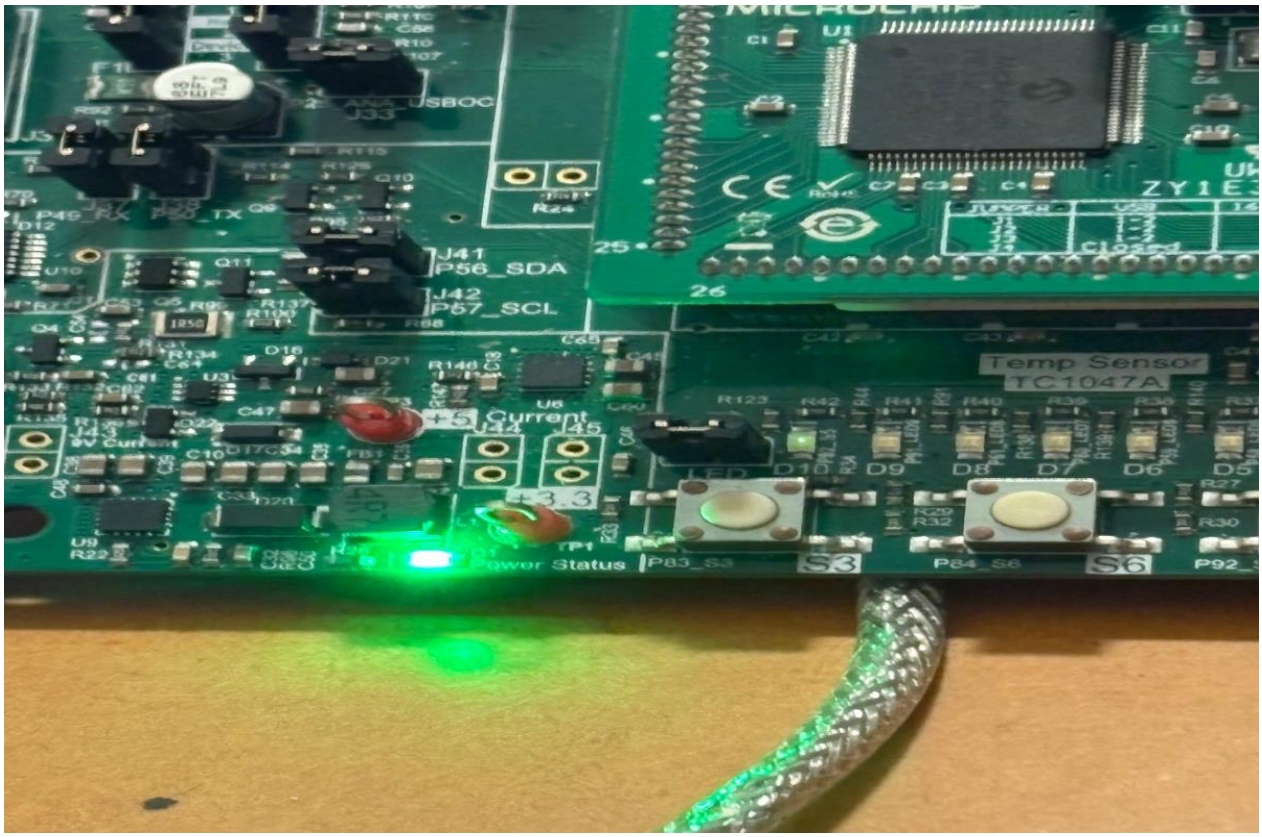


Figure 3:S3 and S6 for navigation in the UI interface

3.2 EEPROM (25LC256)

The EEPROM memory (25LC256) is used to store the last 10 temperature readings. The EEPROM is connected to the microcontroller via the SPI interface, allowing the system to write and read data efficiently.

In this step, we:

- **Configure the SPI interface in MCC to allow communication with the EEPROM.**

  To configure the SPI interface in MPLAB Code Configurator (MCC) for communication with the EEPROM (25LC256), follow these steps:

  - In MCC, go to the "Peripherals" tab and click on "SPI1" .
  - Set the SPI mode to "Master" because the microcontroller will be controlling the communication with the EEPROM.

- Select the appropriate pins for SCK, SDI, and SDO (SPI clock, data in, and data out) according to your hardware connections. You can use the pin manager in MCC to configure these pins.

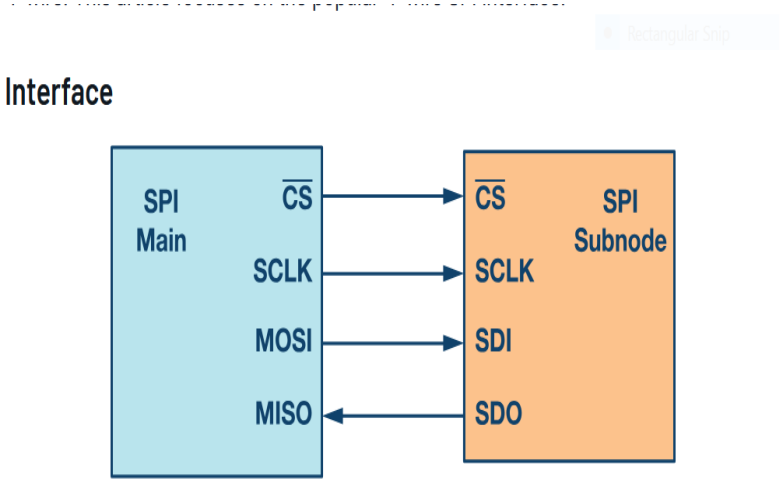


Figure 4:SPI Interface

The configuration of the SPI communication protocol between the microcontroller (UC) and the EEPROM (25LC256) is relatively simple due to the availability of comprehensive documentation for both the UC and the EEPROM. The MPLAB Code Configurator (MCC) simplifies the process by providing a user-friendly interface to configure the SPI module. Key aspects such as selecting the correct SPI mode (Master), setting the baud rate, choosing the data order (MSB First), and configuring the clock polarity and phase are all made straightforward by MCC.

However, the main challenge in this configuration lies in properly linking the Chip Select (CS) pin of the EEPROM to the microcontroller. The CS pin is crucial because it signals the EEPROM to begin communication, and ensuring that this pin is correctly configured to the microcontroller's output port is essential for reliable data transfer.

While the rest of the process is facilitated by the available documentation and tools, such as the SPI configuration in MCC, the CS pin connection requires careful attention to ensure it is correctly mapped to the appropriate port and configured as an output. This is typically where most issues arise, but once the CS pin is properly configured, the communication between the microcontroller and the EEPROM should work seamlessly.

- **Develop functions to store the temperature readings into the EEPROM memory. Each reading is saved with its corresponding timestamp, forming a complete record of the measurement.**

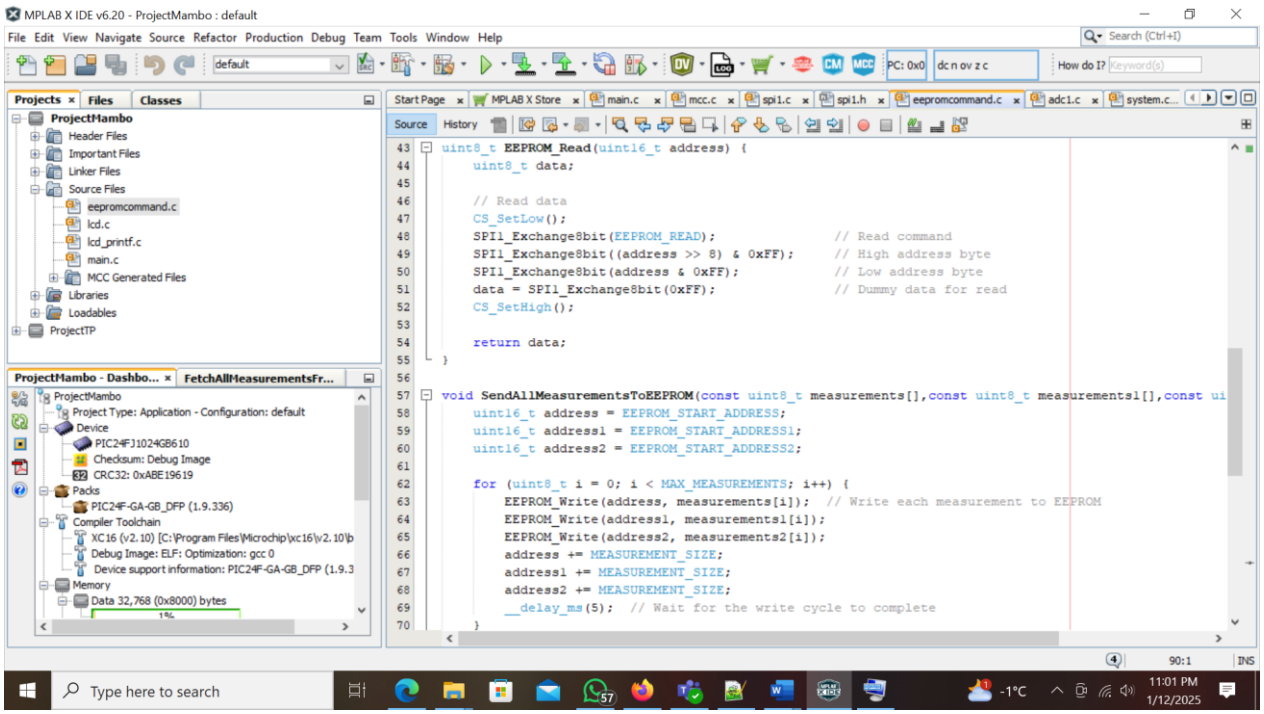


Figure 5: Link between EEPROM and micro-processor

In this part of the project, after establishing the SPI communication protocol with the EEPROM using the MCC-generated code, I created a new file called **EEPROMCommand.c.** This file links the SPI communication code with the temperature readings and timestamps, enabling the system to save the measurements to the EEPROM memory.

Explanation:

1. **Efficient Use of EEPROM**:
   The data is stored in a circular manner across different memory addresses (EEPROM_START_ADDRESS, EEPROM_START_ADDRESS1, and EEPROM_START_ADDRESS2) to accommodate up to MAX_MEASUREMENTS. This design ensures that the EEPROM can store a maximum of 10 measurements, and new measurements can overwrite old ones if necessary.

2. **User Interface Integration**:
   The functions SendAllMeasurementsToEEPROM and FetchAllMeasurementsFromEEPROM provide the mechanism for writing and reading the data, respectively. These functions are then linked to the user interface (UI), which allows the user to view the stored temperature readings via a menu system on the LCD.

By using these functions, I was able to seamlessly integrate temperature data storage into the EEPROM, enabling historical measurement retrieval as part of the overall project.

- **Implement functionality to read the last 10 temperature readings from the EEPROM and display them when requested by the user.**

The EEPROM has a limited number of write cycles, so proper handling of memory writes (e.g., using a circular buffer or overwriting the oldest reading when the memory is full) is important to ensure efficient memory use.

3.3 Temperature Measurement and Storage

This part involves integrating the temperature measurement process with the EEPROM storage mechanism. Each time a temperature reading is taken, the following steps occur:

- **The temperature reading is acquired using the ADC (as described in Part 1).**

```
if (flag){
    ADC1_ConversionResultBufferGet( &buffer);
    temperature = (((buffer * 3.3) / 1023)-0.5)*100;
    StoreTemperature(temperature * 10);

    if (historyIndex == 0) {
        SendAllMeasurementsToEEPROM(temperatureHistory,secondHistory,minuteHistory);
```

Figure 6: Temperature measurement

- **The reading is stored in the EEPROM, along with its timestamp from the RTCC (as described in Part 2).**

```
84
85  void StoreTemperature(uint16_t temperature) {
86      // Store the temperature in the current index
87      temperatureHistory[historyIndex] = temperature;
88      RTCC_TimeGet( &currentTime);
89      minuteHistory[historyIndex] = currentTime.tm_min;
90      secondHistory[historyIndex] = currentTime.tm_sec;
91      // Increment the index and wrap around if needed
92      historyIndex = (historyIndex + 1) % MAX_MEASUREMENTS;
93  }
```

Figure 7: Saving locally and then using spi to store on EEPROM

- **Once the memory is full (i.e., after 10 readings), the system starts overwriting the oldest readings to maintain a constant record of the last 10 measurements.**

```
if (historyIndex == 0) {
    SendAllMeasurementsToEEPROM(temperatureHistory,secondHistory,minuteHistory);
```

Figure 8:Maintain constant record

By using the EEPROM for storage, the system can retain data even when the microcontroller is powered off, providing long-term storage for temperature readings.

Challenges

- Memory Management: Efficiently managing the EEPROM to store and retrieve the last 10 temperature readings is key. This can be done by utilizing circular buffers or by overwriting older data once the memory limit is reached.
- SPI Communication: Configuring the SPI interface correctly to handle EEPROM read/write operations and ensure reliable data storage is crucial. Potential issues include timing and synchronization problems with the EEPROM chip**.**
- UI Navigation: Ensuring the UI is intuitive and easy to use for retrieving historical data may require careful design and testing.

However, as this process builds on concepts already explored in class (such as SPI communication and data storage), it was relatively easy to integrate these components into the system.

Once all the connections between the various parts of the project were completed—specifically, integrating the temperature sensor, SPI communication with the EEPROM, and the LCD display—I encountered an issue where the temperature sensor began producing unstable values. This problem was unexpected, as the sensor had been functioning correctly during earlier standalone testing.

Potential Causes of Instability:

1. Noise or Interference:
   The instability in the temperature readings was likely caused by electrical noise or interference in the circuit. With the SPI communication, LCD, and temperature sensor all connected, the shared power supply or ground plane might have introduced noise that affected the analog signal from the sensor.
2. Improper ADC Configuration:
   Although the ADC settings were configured earlier, the additional components might have affected the accuracy of the ADC readings, such as incorrect sampling times or an insufficiently stable reference voltage.
3. Power Supply Issues:
   The increased load from all connected components may have caused voltage fluctuations, affecting the sensor's performance.

II. General Conclusion

This project served as a comprehensive exercise in integrating multiple components—such as a temperature sensor, EEPROM memory, and an LCD display—into a cohesive embedded system. Each part of the project presented unique challenges, from configuring the ADC module for accurate temperature measurement to implementing SPI communication for data storage. By systematically addressing these challenges, I was able to build a functional system capable of measuring, storing, and displaying temperature readings alongside their corresponding timestamps.

The project also emphasized the importance of adhering to a structured approach in designing embedded systems, especially when dealing with real-world constraints like unstable sensor readings or communication issues. Ultimately, it provided a practical understanding of how to integrate hardware and software efficiently in microcontroller-based applications.

1. Lessons Learned

Throughout this project, I gained valuable technical and problem-solving skills that can be summarized as follows:

A. **Step-by-Step Development**

- Breaking down the project into smaller, manageable tasks allowed me to focus on each component individually.
- For instance, I began by configuring the ADC for temperature readings, then integrated the RTCC module, and finally implemented EEPROM storage.

B. **Unity Testing for Each Module**

- Before integrating components, I conducted unit tests on each module to ensure they functioned independently.
- Example: Testing the ADC module by manually displaying temperature values on the LCD before adding EEPROM functionality.

C. **Debugging Techniques**

- Debugging was critical for identifying and resolving issues during integration.
- Tools such as MPLAB X's watch window, breakpoints, and real-time variable monitoring were instrumental in pinpointing errors.

- Example: Identifying the source of unstable temperature readings by isolating the sensor and testing it separately from the rest of the system.

D. **Systematic Problem Solving**

- When faced with the challenge of unstable sensor readings, I applied a systematic approach to resolve the issue:
  - Verified hardware connections.
  - Adjusted ADC sampling parameters.
  - Added decoupling capacitors to reduce noise.

E. **Efficient Code Organization**

- I learned to modularize the code, such as creating a dedicated file (eepromcommand.c) for SPI-based EEPROM commands, making the project easier to maintain and extend.

F. **Importance of Documentation and Datasheets**

- Leveraging datasheets for the TC1047A sensor, EEPROM, and PIC24FJ1024GB610 microcontroller was vital for correct configuration and troubleshooting.
- Example: Using the EEPROM datasheet to implement SPI commands and manage memory addressing.

G. **Real-World Challenges**

- Working with a complete embedded system highlighted the real-world challenges of hardware-software integration, such as managing power supply stability and timing issues.

2. Final Takeaway

This project demonstrated the importance of structured workflow, thorough testing, and systematic debugging in embedded systems development. By addressing each challenge step by step and using available tools and documentation, I was able to build a robust and functional system, while also gaining confidence in my ability to tackle similar projects in the future.